INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Adaptive Web Penetration Testing with Q-Learning and Deep Q-Networks on OWASP Juice Shop

Kevin Imanuel Setiawan

Bachelor's Program in Information Technology, Telkom University, Surabaya, Indonesia

* Corresponding author: kevinit@student.telkomuniversity.ac.id

Abstract: This study compares tabular Q-learning (QL) and a deep Q-network (DQN) for adaptive web penetration testing on OWASP Juice Shop. Both agents operated under a fixed budget of 1,585 episodes over 120 endpoint-payload actions. Candidate findings were quality-gated using control-differential probes, while cross-site scripting candidates also required execution in a browser. QL produced 14 verified findings and DQN produced 12; their first findings appeared in episode 1, and each reached its final observed finding count by episodes 787 and 443, respectively. A separate detector benchmark yielded 13 true positives, no false positives, and one false negative among labeled actions, with 46 actions left unknown. A scoped comparison on two shared endpoints recorded 10 QL findings and two relevant ZAP findings, but the request procedures were not equivalent. The results demonstrate feasibility in this environment, not generalization to other applications.

Keywords: web penetration testing; reinforcement learning; Q-learning; deep Q-network; OWASP Juice Shop; OWASP ZAP

1. INTRODUCTION

Web applications expose multiple input points and behaviors, so penetration testers must choose requests under limited time and request budgets. Reinforcement learning offers a way to learn action preferences from observed responses. Q-learning (QL) estimates action values in a table, whereas a deep Q-network (DQN) approximates values with a neural network [1]–[3]. Prior research has explored reinforcement learning for penetration testing and capture-the-flag environments, while emphasizing trade-offs between model-free learning and prior knowledge [4].

This study evaluates QL and DQN on OWASP Juice Shop, a deliberately vulnerable web application for security training [5]. We compare verified findings under a fixed episode budget and describe a scoped QL comparison with OWASP ZAP. Findings pass control-differential checks; XSS candidates additionally require browser execution. The evaluation is specific to the target and action set and does not establish generalization to arbitrary websites.

2. MATERIALS AND METHODS

2.1 Test Environment and Action Space

The target was OWASP Juice Shop version 20.2.0 running locally. Eight configured endpoints and 15 payloads formed 120 endpoint-payload actions: six SQL injection (SQLi), four cross-site scripting (XSS), and five fuzzing probes. Endpoint surface labels and payloads were manually curated from knowledge of the application. This human prior narrows the search space, so the system is not fully autonomous. The state comprises HTTP status, response length, error, reflection and vulnerability indicators, endpoint index, response-time category, and content type. DQN receives a 28-dimensional one-hot encoding.

2.2 Agent Configuration and Training Protocol

Parameter	Q-learning	DQN
Episode budget	1,585	1,585
Learning rate	0.1	0.001
Discount factor	0.9	0.9

Exploration	ε: 1.0 to 0.1; decay 0.90	ε: 1.0 to 0.1; decay 0.9965
Value initialization	5.0	Neural network
Hidden units	—	64
Batch / replay	—	32 / 10,000
Target update	—	Every 10 updates
Random seed	42	42

TABLE 2. AGENT CONFIGURATION

Each agent used 1,585 episodes and random seed 42. QL used learning rate 0.1, discount factor 0.9, epsilon from 1.0 to 0.1 with decay 0.90, and optimistic initial values of 5. DQN used a 64-unit hidden layer, learning rate 0.001, batch size 32, replay capacity 10,000, target update interval 10, and epsilon decay 0.9965. Exploration schedules differed to avoid DQN reaching its exploration floor too early. There was no early stopping. Only verified vulnerable actions were masked; previously attempted or failed actions remained available. A benign hello probe established a separate response baseline for each endpoint. Rewards were +50 for verified findings, +10 for anomalies or inconclusive candidates, −1 for normal responses, and −5 for errors, with a configured response-length bonus of 0, 0, 5, or 10.

2.3 Finding Verification and Detector Benchmark

HTTP candidates were checked against a benign control and, when applicable, a neutralized-payload control; a finding was accepted only if neither reproduced its vulnerability class. Connection errors did not count as verification. XSS was evaluated in headless Chromium on the corresponding single-page application route; acceptance required execution of an instrumented canary, with a warmup navigation and bounded polling. A separate detector benchmark evaluated all 120 actions against curated labels. It used the browser path for XSS and excluded 46 unknown actions from binary metrics. Because ground-truth confirmation and detector evidence partly share mechanisms, this is not a fully independent oracle.

The analyzer also applied endpoint-surface constraints when assigning vulnerability classes. SQLi evidence required a database-error response or a sufficiently changed successful response relative to the endpoint baseline. File-disclosure labels were restricted to the configured file surface and required recognizable content or the configured extension-bypass evidence. Fuzzing findings required an echoed probe in a server-error response. These constraints reduce generic error-page and reflection-only findings, but their accuracy remains bounded by the configured signatures and curated surface labels.

The control-differential step addresses false positives by asking whether a benign request or a less exploit-like variant produces the same evidence. Payloads for which neutralization would make no meaningful change used only the benign control. An inconclusive candidate was retained as an anomaly reward rather than counted as a verified vulnerability. The browser check serves a separate purpose: it distinguishes data reflected in an HTTP response from a script that actually executes in the rendered application.

2.4 ZAP Comparison and Metrics

The controlled ZAP comparison was restricted to the two endpoints shared with QL: product search and the FTP surface. ZAP provides automated scanning and alert reporting [6]. We report verified finding count, hit rate (findings divided by training episodes), payload requests per finding, first-finding episode, episode at which the final observed finding count was reached, and unique endpoint–class locations. These are in-sample training results: no held-out test set was used. The endpoint scope is shared, but scan strategy, payload selection, verification overhead, and request accounting differ between QL and ZAP.

3. RESULTS AND DISCUSSION

3.1 QL and DQN Training Results

QL produced 14 verified findings and DQN produced 12 (Table 3). Both found a vulnerability in episode 1. QL reached its final observed count by episode 787, whereas DQN did so by episode 443. These values describe accumulation during the recorded run and do not establish convergence or unseen-target performance. QL had more findings, while DQN accumulated a larger total reward, which reflects reward across all episodes rather than finding count alone.

TABLE 3. TRAINING OUTCOMES REPORTED IN THE THESIS

Measure	QL	DQN
Episodes	1,585	1,585
Verified findings	14	12
Categories	6 SQLi; 5 XSS; 3 fuzz/file	6 SQLi; 4 XSS; 2 fuzz/file
Hit rate	0.88%	0.76%
Requests/finding	113.21	132.08
First finding	Episode 1	Episode 1
Final count reached	Episode 787	Episode 443
Unique locations	5	5
Elapsed time	154.23 s	160.04 s

Total reward	3,785	10,209

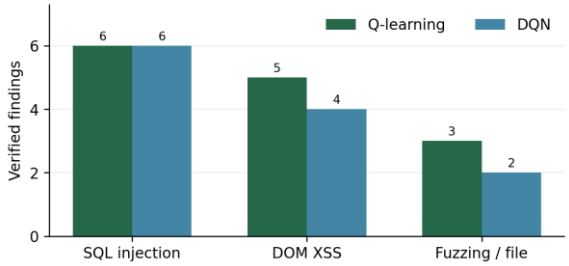


Fig. 1. Verified finding categories reported for each agent.

The category counts show six SQLi findings for each agent. QL reported one additional XSS and one additional fuzzing/file finding. Multiple payload findings at a single endpoint and class contribute one unique location; these results are not an inventory of independent production vulnerabilities.

3.2 QL Endpoint Activity

The difference between reward and verified-finding totals follows from the reward design. A vulnerability contributes a large positive reward, but the final cumulative score also includes normal responses, errors, anomalies, and response-length bonuses throughout all episodes. DQN can therefore obtain a higher cumulative reward while producing fewer distinct accepted findings. Reward should be interpreted as an optimization signal used by each agent, not as a direct substitute for vulnerability coverage.

Product search accounted for most QL requests and findings. The FTP surface produced three findings from 39 requests; track-order produced three; login produced one. Other configured endpoints produced no verified findings in this run. The distribution reflects the curated action space and should not be interpreted as a complete vulnerability inventory.

TABLE 4. QL REQUESTS AND VERIFIED FINDINGS BY ENDPOINT

Endpoint	Requests	Findings
Product search	1,184	7
FTP surface	39	3
Product reviews (GET)	24	0
Track order	252	3
User login	24	1
Product reviews (PUT)	21	0
Feedback	22	0
Challenges	19	0
Total	1,585	14

3.3 Detector Benchmark

The labeled benchmark contained 13 true positives, no false positives, one false negative, and 60 true negatives; 46 actions remained unknown. Precision was 1.000, recall 0.929, F1 0.963, and specificity 1.000. The false negative was the SVG XSS action on the track-order route. Zero false positives applies only to the 60 labeled negatives and does not establish perfect performance on untested applications or payloads.

For clarity, precision is TP/(TP+FP), recall is TP/(TP+FN), F1 is the harmonic mean of precision and recall, and specificity is TN/(TN+FP). Here these measures are computed only on the benchmark's labeled subset. The 46 unknown actions are not assumed safe and are not included as negatives; this prevents unknown labels from artificially increasing specificity.

TABLE 5. DETECTOR PERFORMANCE ON LABELED ACTIONS

Metric	Value
True positives	13
False positives	0
False negatives	1
True negatives	60
Unknown actions	46
Precision	1.000
Recall	0.929
F1-score	0.963
Specificity	1.000

3.4 Scoped Comparison with OWASP ZAP

For the two shared endpoints, the thesis reports 1,223 QL payload requests for 10 findings (122.30 requests per finding). The controlled ZAP scan reports 1,585 requests and two relevant findings (792.50 requests per finding). The ratio of these rates is approximately 6.48. ZAP produced 32 raw alerts, of which two were treated as relevant. This is endpoint-scope matching, not a fully procedure-matched apples-to-apples comparison: request workflow, payloads, verification overhead, and counting rules differ. The 6.48 ratio is descriptive for this experiment and cannot support a general efficiency claim.

TABLE 6. COMPARISON ON TWO SHARED ENDPOINTS

Method	Requests	Relevant findings	Requests/finding
QL selected-payload subset	1,223	10	122.30
Controlled ZAP scan	1,585	2	792.50

3.5 Limitations

The evaluation used one target and one seed per algorithm; training results are in-sample. Endpoint labels and payloads encode human knowledge. The detector benchmark shares evidence with its verification oracle, and the ZAP comparison does not align all scan procedures. Future work should repeat runs across seeds, use held-out targets or actions, obtain independent manual validation, and equalize request and time budgets for baseline comparisons.

4. CONCLUSION

Under 1,585 episodes on OWASP Juice Shop, QL obtained 14 verified findings and DQN obtained 12. DQN reached its final observed finding count earlier, while accumulating greater total reward. The detector benchmark agreed strongly with curated labels, subject to partial dependence on shared verification evidence. The scoped ZAP comparison favored QL in requests per finding, but procedural differences make that result descriptive. Repeated and held-out evaluations are required before drawing broader conclusions.

CONFLICT OF INTEREST

The author declares no conflict of interest. (Please confirm this declaration before submission.)

REFERENCES

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[3] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[4] F. M. Zennaro and L. Erdodi, "Modeling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs

between model-free learning and a priori knowledge," arXiv:2005.12632, 2020.

[5] OWASP Foundation, "OWASP Juice Shop." [Online]. Available: https://owasp.org/www-project-juice-shop/. Accessed: Oct. 9, 2026.

[6] OWASP Foundation, "ZAP Automation Framework." [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/automation-framework/. Accessed: Oct. 9, 2026.